DOMS | Page No.
Date / /

* Summary of changes:

1. SQL operator precedence [SQL / backend / Oracle]: Added parantheses around the title / description OR in TaskRepository.db/queries/search_tasks.sql, and both queries in the Oracle package. Archived tasks no longer leak into results and the status filter now applies to every match. Added id Desc as an ORDER BY tiebreaker for stable pagination.

2. Removed Thread.sleep in TaskController: Shorter queries were slower (up to 1s), which also made out-of-order response likely.

3. useTasks hook: Added effect cleanup so outdated responses are ignored, cleared the error on each new request, and reset loading in finally so errors actually display.

4. Frontend UX: Reset to page 1 when search or status changes, debounced search by 300ms.

DOMS | Page No.
Date / /

5. Input validation: An invalid status now returns 400 instead of 500. page and pageSize are clamped (page >= 1, pageSize 1-100).

* What I chose not to change.

a. In-memory pagination (loads all rows, then sublist): a proper fix needs Pageable plus a count query, which is a bigger diff than a patch exercise calls for.
b. LIKE wildcard escaping (%, _), debug logging cleanup, the Oracle VARCHAR(257) limit and ROWNUM style: low impact.
c. H2 console and hardcoded CORS origin: fine for local use.
d. Returning the entity instead of a DTO: fine at this size.

* Biggest remaining risk:

Pagination happens in application memory. Every request loads all matching rows, so latency and memory grow with table size. Moving LIMIT / OFFSET (or Pageable) into the query is the next fix.

DOMS | Page No.
Date / /

* Tools / AI used :

I used claude to read through the codebase and produce a ranked list of suspected issues with suggested fixes. I then ran the app, reproduced each bug myself, applied and tested the fixes, and wrote the handwritten explanations in my own words.